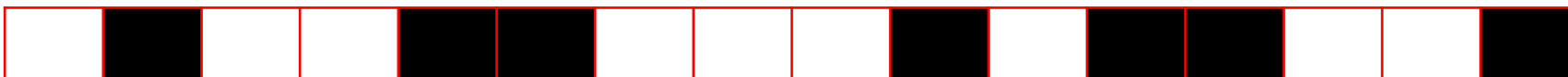




D&C CLASS WORK

WHITE-BLACK PAIR



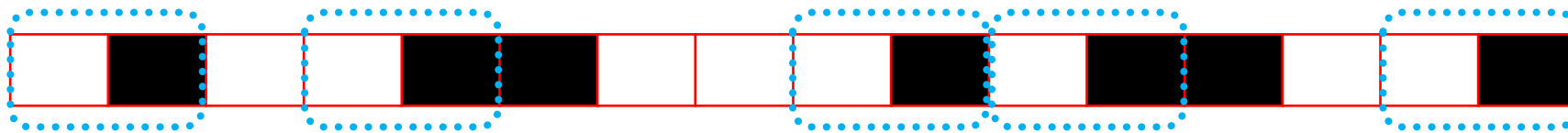
You are given an array A where each cell is colored either white or black, but the colors are hidden. You are allowed to ask queries to reveal the color of a cell. It is known that $A[0]$ is white and $A[n - 1]$ is black.

Find a white-black pair by revealing the minimum number of cells.

WHITE-BLACK PAIR



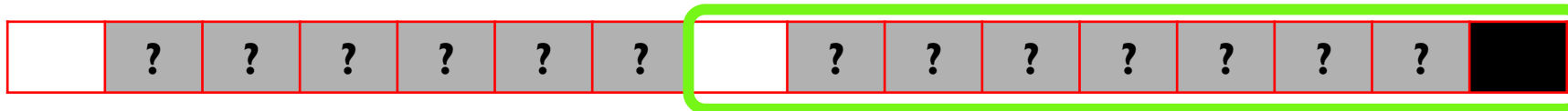
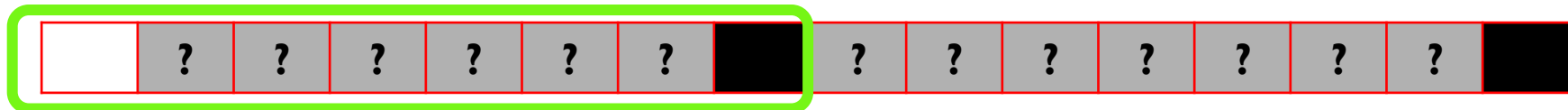
WHITE-BLACK PAIR



A white cell followed by a black cell in this array is called a white-black pair.

You may point to any cell and ask a question “What is its color?”
Your goal is to find a white-black pair by asking the minimum number of questions.

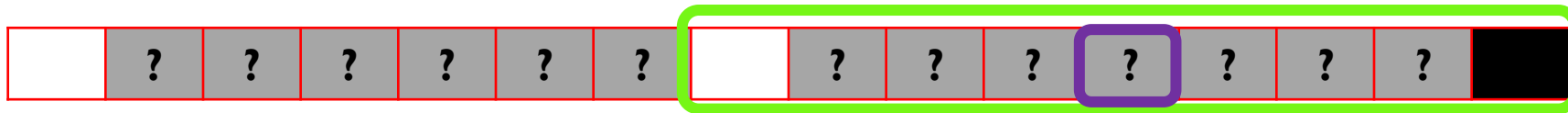
WHITE-BLACK PAIR



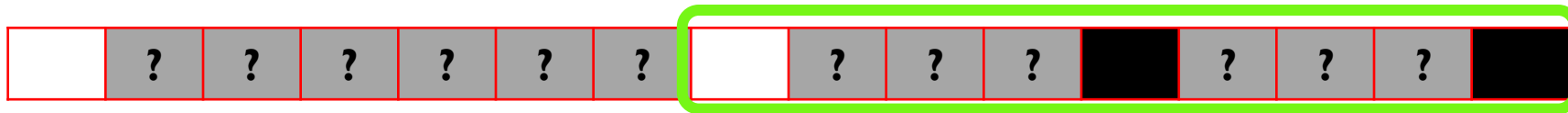
WHITE-BLACK PAIR



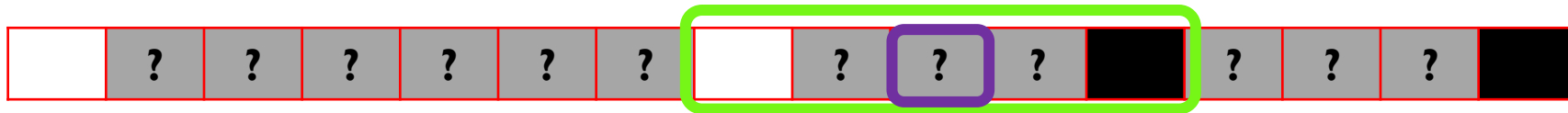
WHITE-BLACK PAIR



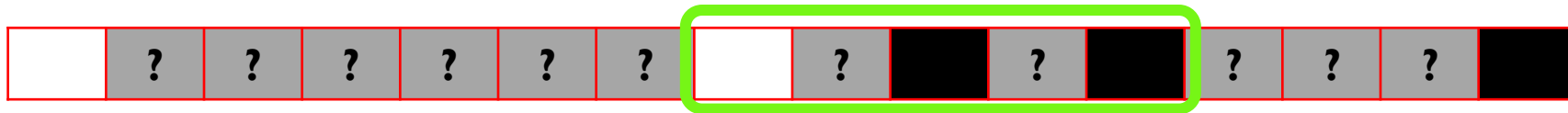
WHITE-BLACK PAIR



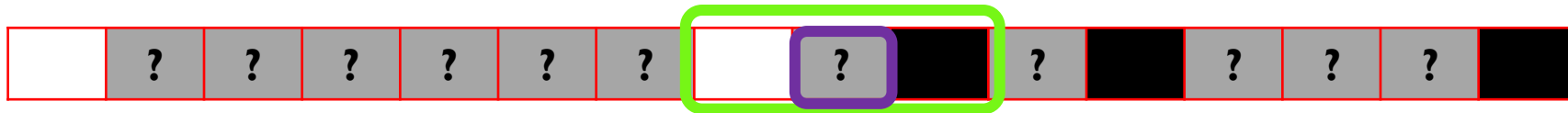
WHITE-BLACK PAIR



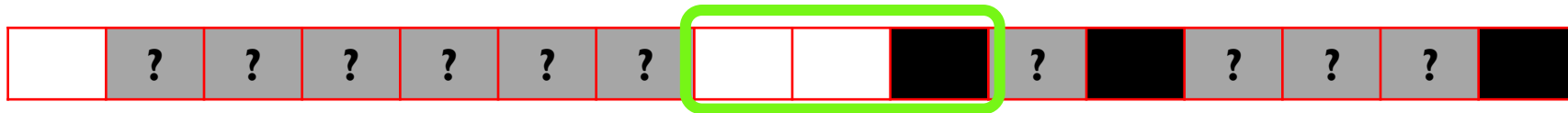
WHITE-BLACK PAIR



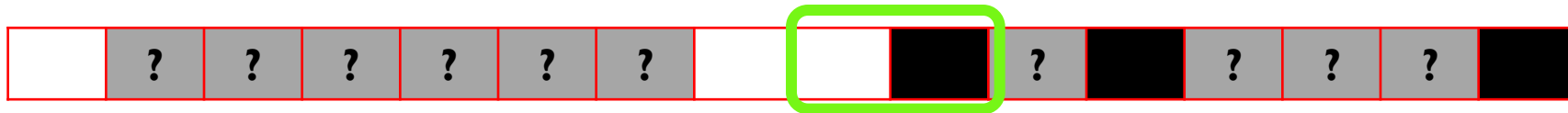
WHITE-BLACK PAIR



WHITE-BLACK PAIR



WHITE-BLACK PAIR



PSEUDOCODE

```
FUNCTION FindPair(A, left, right):  
  
    IF right == left + 1:  
        RETURN (left, right)  
  
    mid = (left + right) // 2  
  
    IF A[mid] == 'white':  
        RETURN FindPair(A, mid, right)  
    ELSE:  
        RETURN FindPair(A, left, mid)
```

FINDING A PEAK

An element of a sequence is called a peak if it is greater than all its neighbors.

7	13	2	8	1	9	6	15	16	10	4	3	12	18	5	11
---	----	---	---	---	---	---	----	----	----	---	---	----	----	---	----

FINDING A PEAK

Find a peak by revealing the minimum number of elements of this sequence.

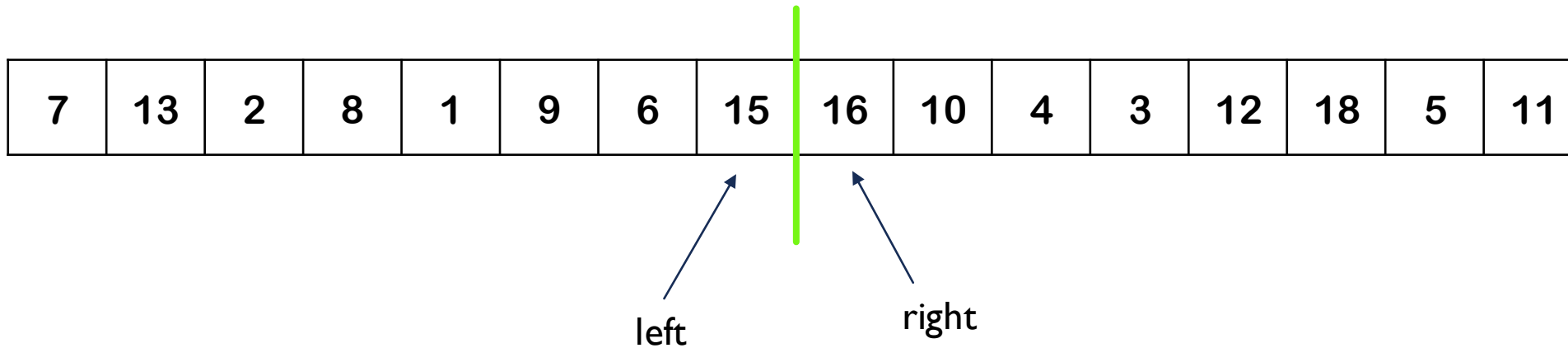
?	?	?	?	?	?	?	?	?	?	?	?	?	?	?	?
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Note that any sequence contains a peak: the largest element is a peak.

FINDING A PEAK

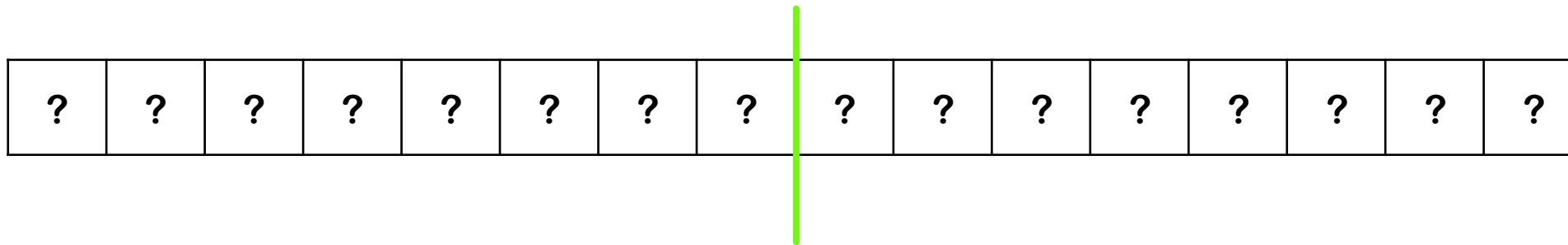
7	13	2	8	1	9	6	15	16	10	4	3	12	18	5	11
---	----	---	---	---	---	---	----	----	----	---	---	----	----	---	----

FINDING A PEAK

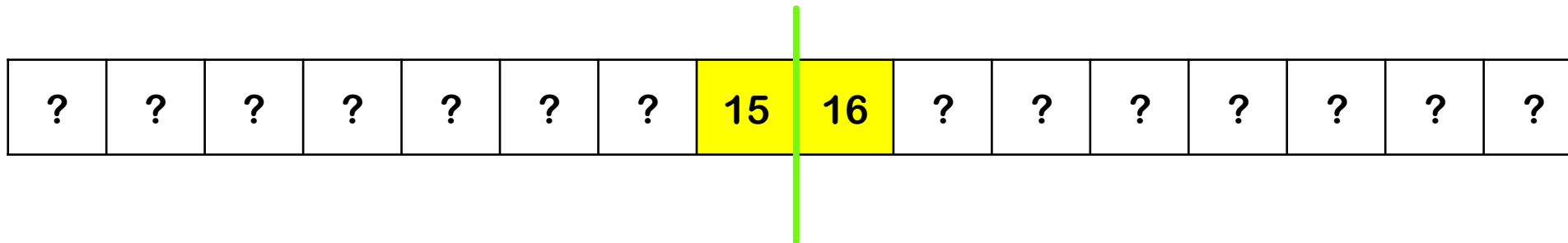


1. $\text{left} > \text{right}$: find the peak at the left part
2. $\text{left} < \text{right}$: find the peak at the right part

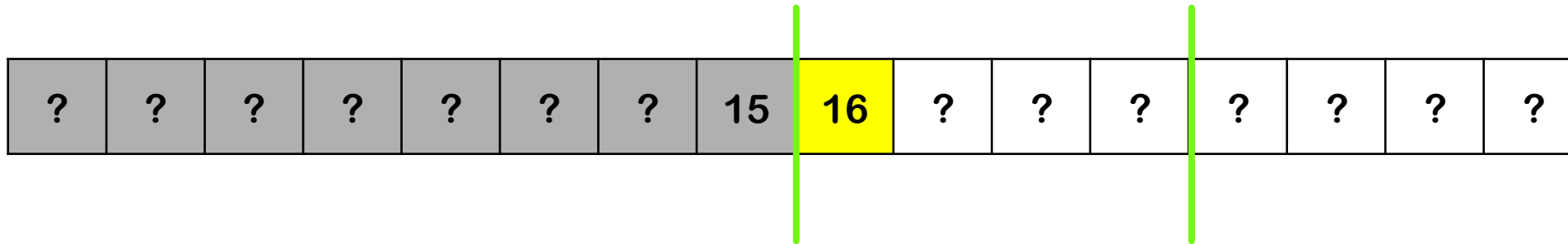
FINDING A PEAK



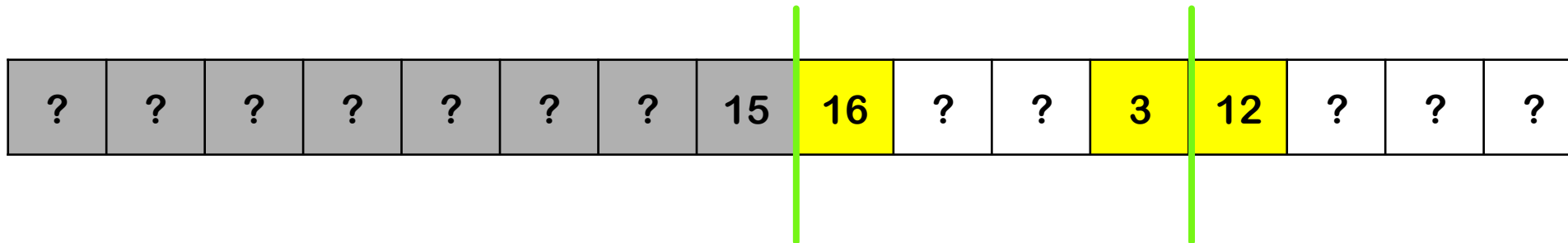
FINDING A PEAK



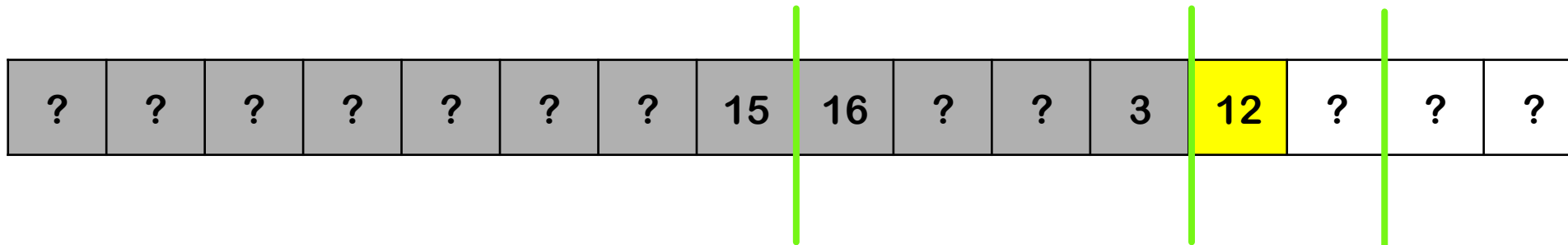
FINDING A PEAK



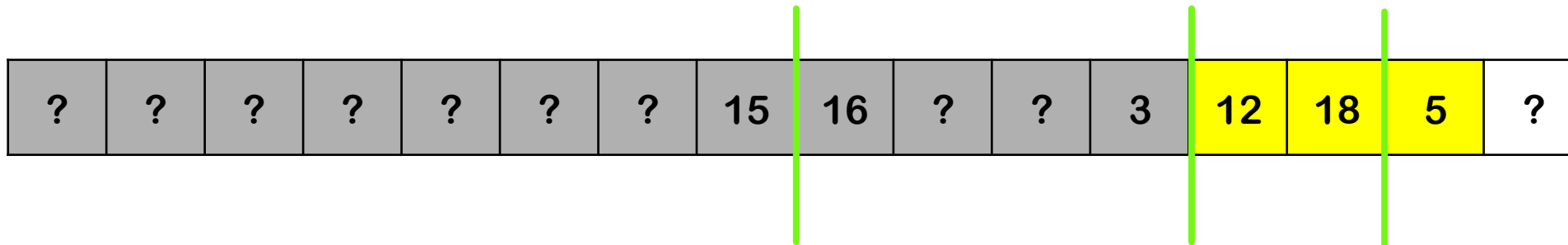
FINDING A PEAK



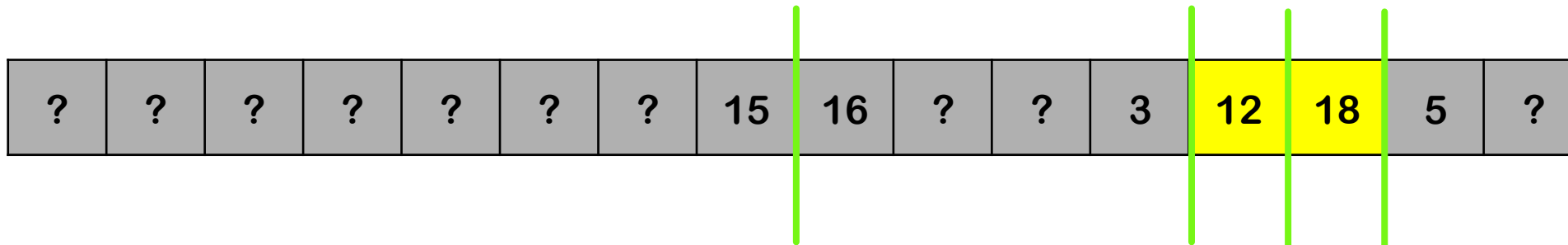
FINDING A PEAK



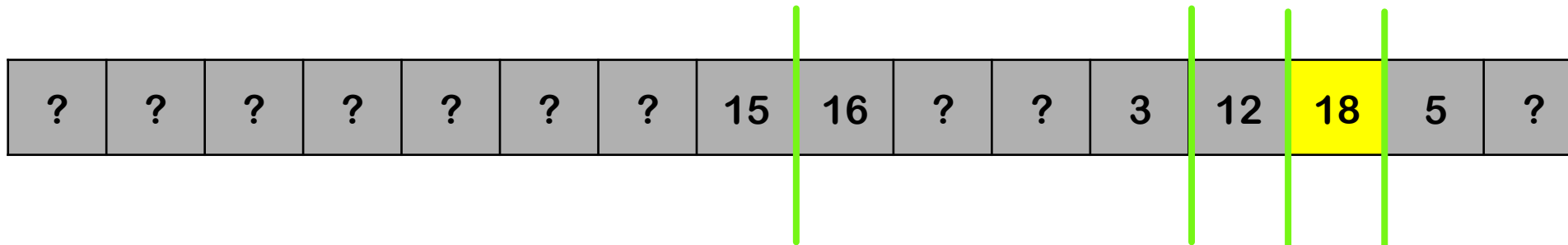
FINDING A PEAK



FINDING A PEAK



FINDING A PEAK

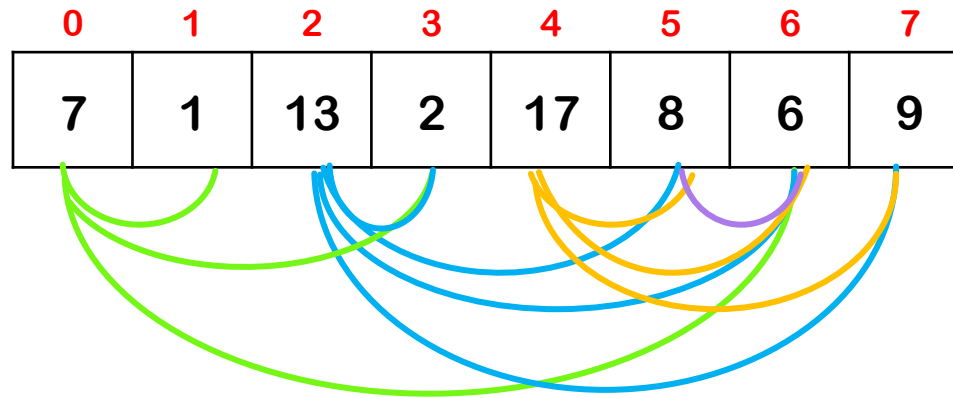


PSEUDOCODE

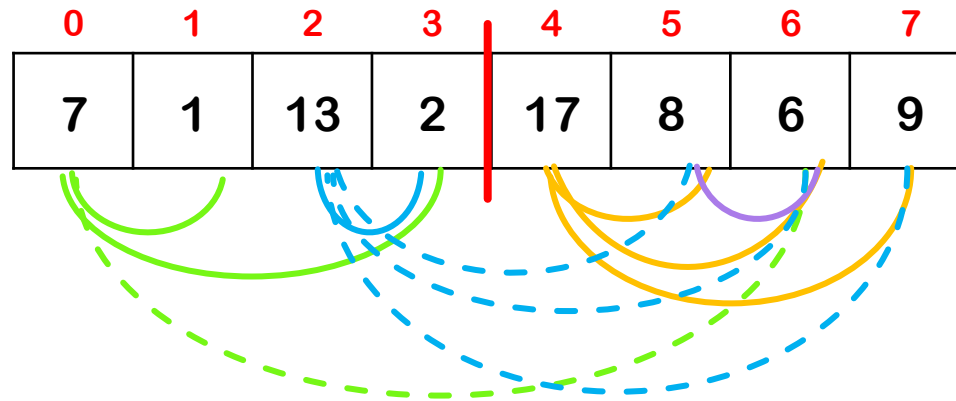
```
FUNCTION FindPeak(A, left, right):  
    IF left == right:  
        RETURN left  
  
    mid = (left + right) // 2  
  
    // Reveal A[mid] and A[mid+1]  
    IF A[mid] < A[mid + 1]:  
        RETURN FindPeak(A, mid + 1, right)  
    ELSE:  
        RETURN FindPeak(A, left, mid)
```

NUMBER OF INVERSIONS

- Compute the number of inversions in a sequence of integers $a_1, a_2, \dots, a_n$: the number of indices $i < j$ such that $a_i > a_j$.

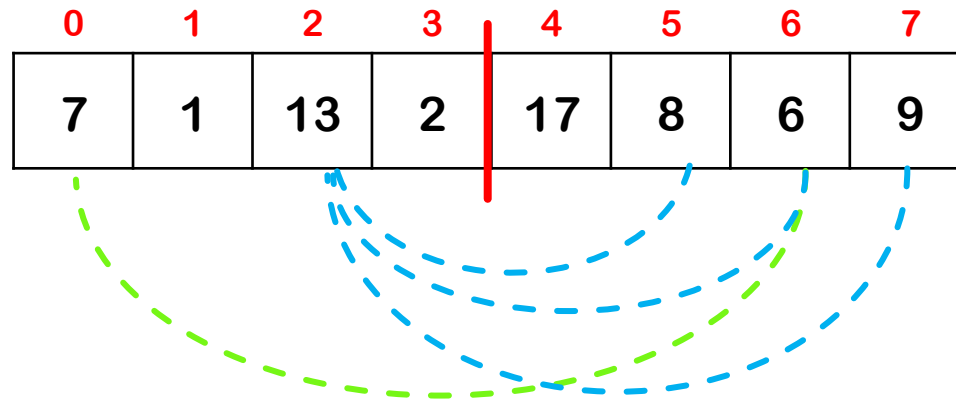


NUMBER OF INVERSIONS



1. Compute the number of inversions in the left side
2. Compute the number of inversions in the right side
3. Compute the number of inversions such that a_i is at the left side and a_j is at the right side

NUMBER OF INVERSIONS



1. Compute the number of inversions in the left side
2. Compute the number of inversions in the right side
3. Compute the number of inversions such that a_i is at the left side and a_j is at the right side

NUMBER OF INVERSIONS

0	1	2	3		4	5	6	7
1	2	7	13		6	8	9	17

3.1. Sort left side

3.2. sort right side

3.3. Count the number of inversions such that a_i is at the left side and a_j is at the right side as merging two halves

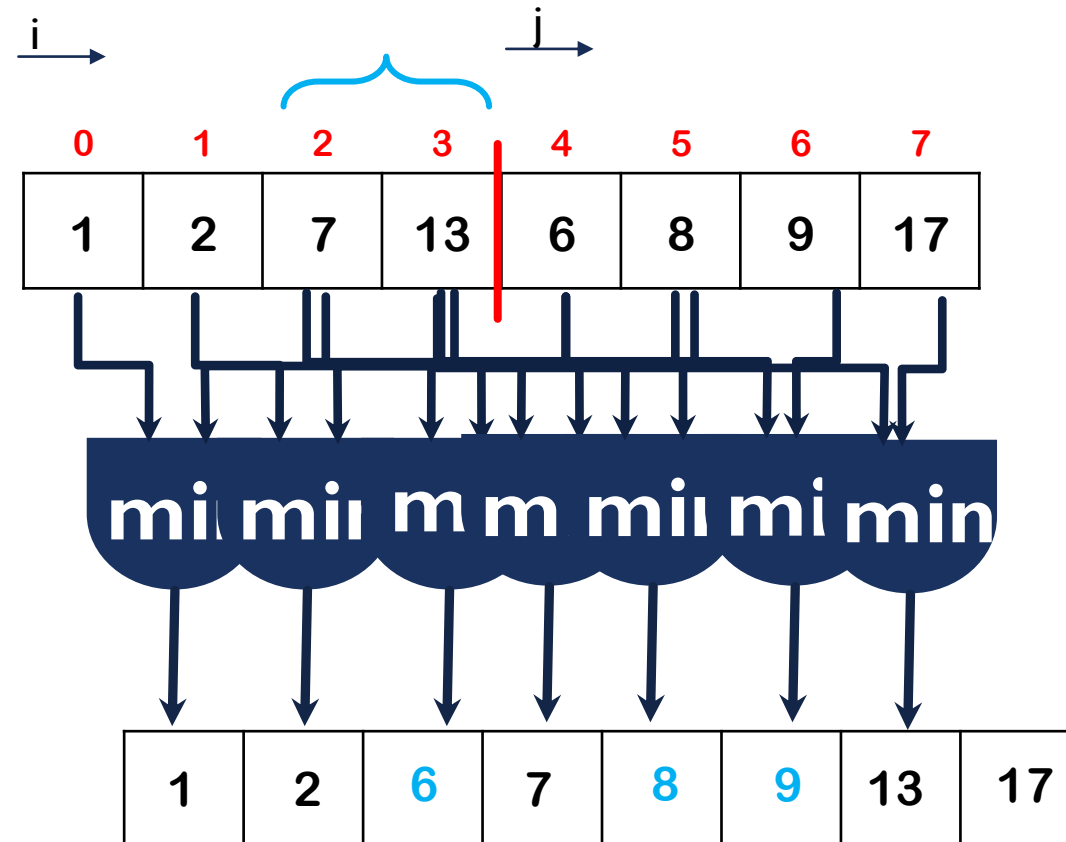
NUMBER OF INVERSIONS

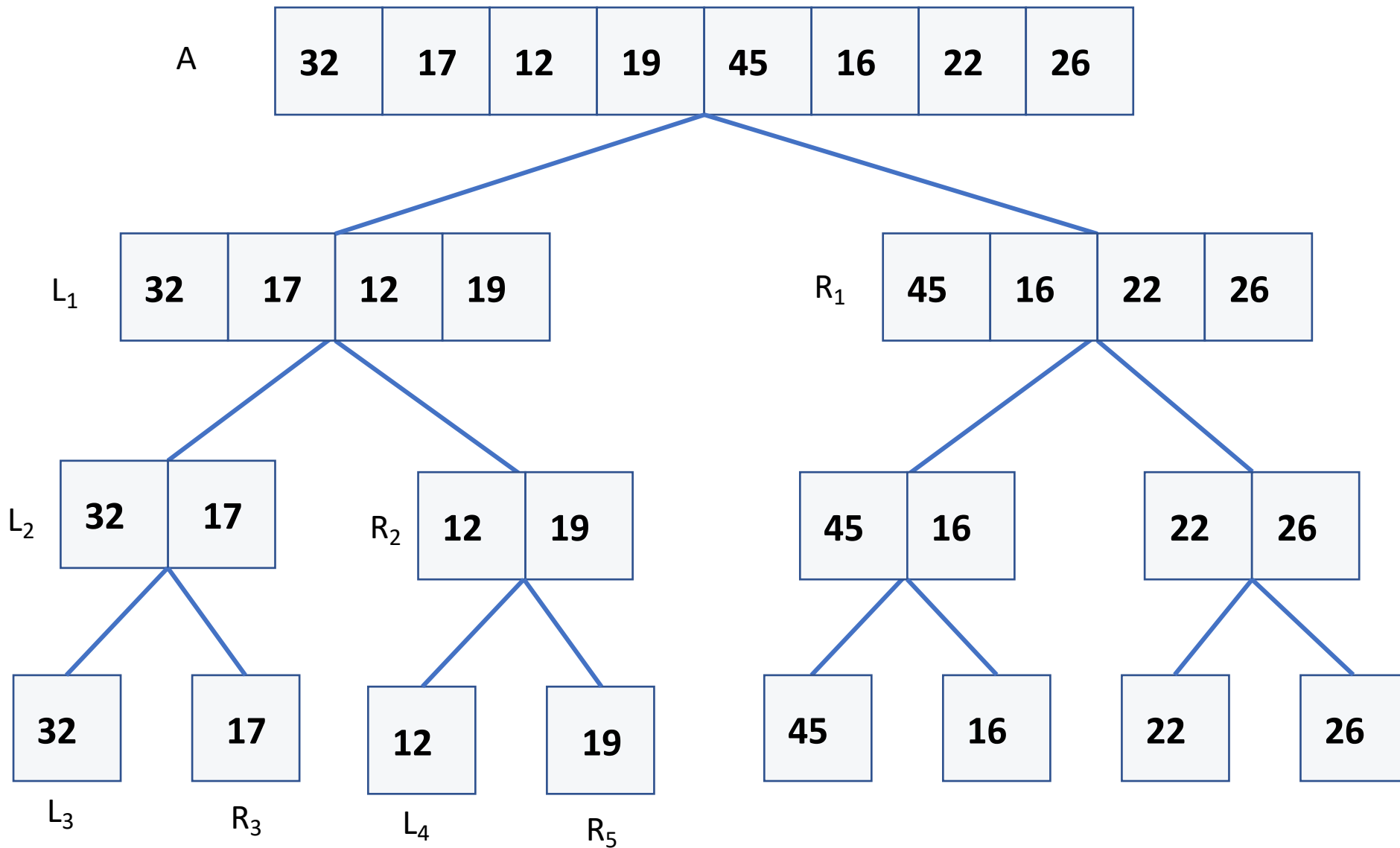
count=0

count+=mid-i+1=2

count+=mid-i+1=3

count+=mid-i+1=4





+3+1+1

A



total=12 inversions

+2+1

L₁



+1+1 R₁



+1

L₂



R₂



+1



L₃

R₃

L₄

R₅

```
FUNCTION CountInversions(A, left, right):  
    IF left >= right:  
        RETURN 0  
  
    mid = (left + right) / 2  
  
    inv_left = CountInversions(A, left, mid)  
    inv_right = CountInversions(A, mid + 1, right)  
    inv_cross = MergeAndCount(A, left, mid, right)  
  
    RETURN inv_left + inv_right + inv_cross
```

```
FUNCTION Merge      (A, left, mid, right):  
    i,j = left, mid + 1  
    temp = empty array  
  
    WHILE i <= mid AND j <= right:  
        IF A[i] <= A[j]:  
            append A[i] to temp  
            i = i + 1  
        ELSE:  
            append A[j] to temp  
  
            j = j + 1  
  
    WHILE i <= mid:  
        append A[i] to temp  
        i = i + 1  
  
    WHILE j <= right:  
        append A[j] to temp  
        j = j + 1  
  
    copy temp back into A[left ... right]
```

```
FUNCTION MergeAndCount(A, left, mid, right):
    i, j = left, mid + 1
    temp = empty array
    inv_count = 0

    WHILE i <= mid AND j <= right:
        IF A[i] <= A[j]:
            append A[i] to temp
            i = i + 1
        ELSE:
            append A[j] to temp
            inv_count = inv_count + (mid - i + 1)
            j = j + 1

    WHILE i <= mid:
        append A[i] to temp
        i = i + 1

    WHILE j <= right:
        append A[j] to temp
        j = j + 1

    copy temp back into A[left ... right]

    RETURN inv_count
```

ANY QUESTION??

HAVE A NICE DAY!